

RENDEZ-VOUS — ONE ROOM, ONE AGENT, ONE ARTIFACT, MANY HUMANS

Work is multiplayer. Agents are single-player.

- Every agent you use today lives in exactly one person's window.
- So the human becomes the integration layer: copy-paste between two chat windows.
- A screenshot forwarded to the other side. “Let me ask the AI and get back to you.”
- The second person never sees the context.
- The deliverable dies inside one private thread.

Four members. Four surfaces. One session.

MEMBER	SURFACE	SENDS	RECEIVES	JOINED BY
Alice	phone — Telegram	text, voice notes, photos	terse replies, voice, files	texting “new”
Bob	laptop — room web view	text	full transcript + the live artifact	scanning a QR
Atlas	local desktop agent — a machine	what only a local machine knows	the same transcript as the humans	the same two endpoints
the inbox	email	reply-to-thread	a per-turn digest, deliverables	replying once

No install. No shared login. No new app. The room models members, not humans — which is why a machine joins the same way a person does.

IT WORKS — LIVE, ON REAL SURFACES

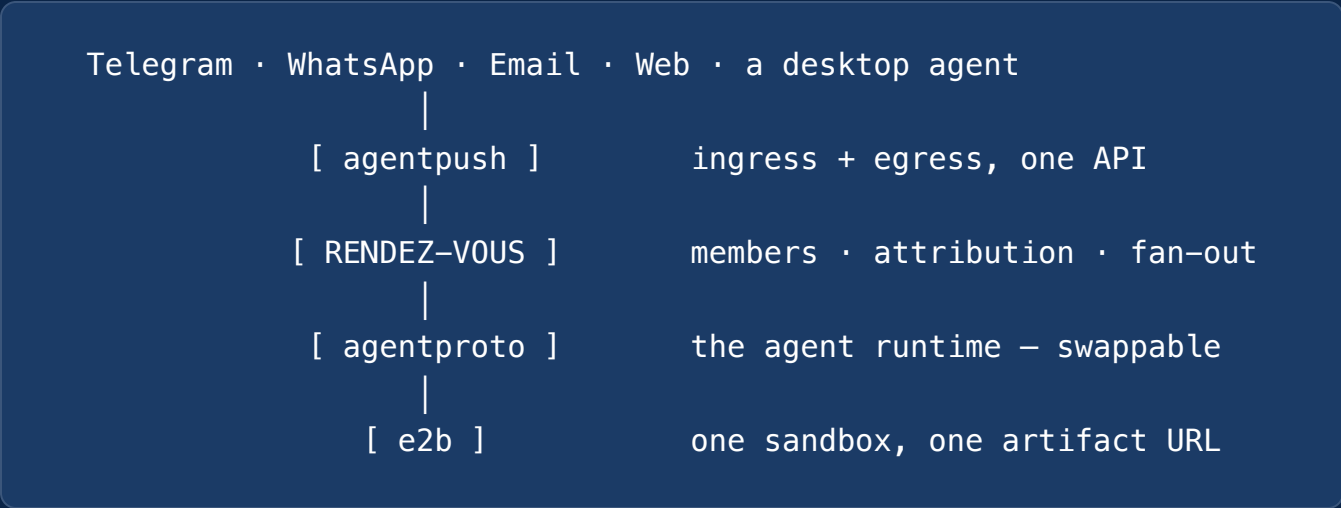
Not a mock. A real phone, a real thread, a real artifact.

0s	A real phone types “new” on Telegram — the room exists
86.5s	Cold boot: agent session and shared artifact both ready
join	A second person joins from a laptop; a desktop agent joins as a member
voice in	A voice note is transcribed and attributed to whoever sent it
image in	A photo is described by the agent and written into the artifact
private	“@me” — one member gets the answer, the others see only that a whisper happened
out	The agent replies with a voice note, and attaches real files
resume	Killed mid-session, the room comes back still knowing what was discussed

Every row observed end to end on live infrastructure, not described.

One pipeline, four layers, one seam that matters.

- agentpush — every channel behind one API, in and out. The room never learns a provider's quirks.
- Rendez-vous — the room: members, attribution, fan-out by presence tier, the authority to send.
- agentproto — our own open-source agent runtime. Unmodified, straight from npm.
- e2b — one sandbox per room, holding the shared artifact. OpenAI for voice and vision, canvakit for the PDF.



The runtime sits under a clean seam. That is the whole claim: swap it and the room does not move.

Built against five silent upstream failures we found and fixed on the way — none of which crashed, all of which lost data or reported success while doing nothing.

A room with a notion of authority, not a relay.

- 1** A member asks the room for the deliverable — a PDF of what everyone just built
- 2** **THE CONFIRMATION GATE**
the room previews it first: recipient, channel, subject, the rendered document — and sends nothing until a member explicitly confirms
- 3** it lands on a phone as a real file, and in an inbox, addressed to someone who was never in the room
- 4** the send is written into the shared transcript: who asked, who confirmed, what, to whom

Every other capability keeps work inside the room. This one lets it leave — under the members' control, on the record.

The room is the primitive.

The runtime is a detail.

One shape, five independent silent failures.

- 1** **FOUND LIVE, ON A PHONE**
a killed session still answers 200 — resume reported success in 0.7s having resumed nothing
- 2** omit queue: true and a message arriving mid-turn is rejected and lost
- 3** the fan-out cursor freezes on resume — every reply after it vanishes
- 4** an account-pinned inbound route is filtered out and never fires
- 5** the agent proposes rebuilding a Gmail capability the workspace already brokered

Four of these lost data quietly. One reported success while doing nothing. None crash; all look like working code or correct configuration.

Each one is fixed, with a regression test. This is why the room records who asked and who confirmed.

What we have not proven.

- Paused sandboxes expire in 20–60 minutes. One did, live, mid-rehearsal — the room advertised a dead link until we made box liveness its own fact.
- WhatsApp is not provisioned: no number, no key. The room is channel-agnostic by construction, so it is a config change, not code.
- We share an existing workspace's Telegram routing; a catch-all route there could fire alongside ours.
- Swapping the runtime for Codex failed at the auth gate, not the swap — a fresh box has no login. The credential model is the real work.